

BANK ACCOUNT MANAGEMENT SYSTEM

Experiment No.	01
Date	11/09/2026
Title	Bank Account Management System using Classes and Objects

1. Objective / Aim

To design and implement a Bank Account Management System in Java using classes and objects, demonstrating object-oriented programming concepts such as classes, objects, constructors, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

2. System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

3. Algorithm / Logic Description

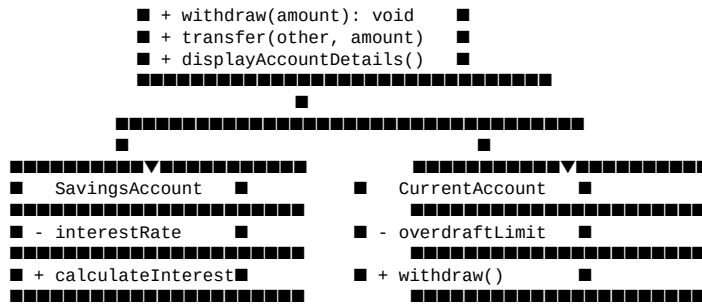
1. Create a base class named Account.
2. Define account number, account holder name, balance, and account type.
3. Create a constructor in Account to initialize account details.
4. Define deposit(), withdraw(), transfer(), and displayAccountDetails() methods.
5. Create SavingsAccount that inherits from Account.
6. Add interestRate and define calculateInterest() to add interest to the balance.
7. Create CurrentAccount that inherits from Account.
8. Add overdraftLimit and override withdraw() to support overdraft.
9. Create SavingsAccount and CurrentAccount objects in main().
10. Perform deposits, withdrawals, interest calculation, and fund transfer.
11. Display account details before and after transactions.
12. Display the final balances.

4. UML Class Diagram

```

classDiagram
    class Account {
        - accountNumber: int
        - accountHolderName: String
        - balance: double
        - accountType: String
        + Account(...)
        + deposit(amount): void
    }

```



Relationship: SavingsAccount and CurrentAccount inherit from Account. CurrentAccount overrides withdraw().

5. Source Code

```

package javapractice;

class Account {
    int accountNumber;
    String accountHolderName;
    double balance;
    String accountType;

    public Account(int accountNumber, String accountHolderName,
        double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited is " + amount);
    }

    void withdraw(double amount) {
        if (balance >= amount) {
            balance = balance - amount;
            System.out.println("Withdrawn is " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    void transfer(Account other, double amount) {
        if (balance >= amount) {
            balance = balance - amount;
            other.balance = other.balance + amount;
            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Insufficient balance for transfer!");
        }
    }

    void displayAccountDetails() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder: " + accountHolderName);
        System.out.println("Account Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}

class SavingsAccount extends Account {
    double interestRate;

    public SavingsAccount(int accountNumber, String accountHolderName,
        double balance, double interestRate) {
        super(accountNumber, accountHolderName, balance, "Savings");
        this.interestRate = interestRate;
    }

    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance = balance + interest;
        System.out.println("Interest Added: " + interest);
    }
}

class CurrentAccount extends Account {
    double overdraftLimit;

    public CurrentAccount(int accountNumber, String accountHolderName,
        double balance, double overdraftLimit) {
        super(accountNumber, accountHolderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }
}
  
```

```

@Override
void withdraw(double amount) {
    if (balance + overdraftLimit >= amount) {
        balance = balance - amount;
        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Overdraft limit exceeded");
    }
}
}

public class BankAccounts {
    public static void main(String[] args) {

        SavingsAccount savings =
            new SavingsAccount(101, "riya", 10000, 5);

        CurrentAccount current =
            new CurrentAccount(102, "Rahul", 5000, 3000);

        System.out.println("befor transactions");

        System.out.println("\nSavings Account:");
        savings.displayAccountDetails();

        System.out.println("\nCurrent Account:");
        current.displayAccountDetails();

        System.out.println("savings account transactions");
        savings.deposit(2000);
        savings.withdraw(1000);
        savings.calculateInterest();

        System.out.println("current account transactions");
        current.deposit(2000);
        current.withdraw(9000);

        System.out.println("amount transfer");
        savings.transfer(current, 3000);

        System.out.println("after transaction");

        System.out.println("\nSavings Account:");
        savings.displayAccountDetails();

        System.out.println("\nCurrent Account:");
        current.displayAccountDetails();

        System.out.println("final amount");

        System.out.println("Savings Account Balance: "
            + savings.balance);

        System.out.println("Current Account Balance: "
            + current.balance);
    }
}

```

6. Compilation & Execution Commands

File name: BankAccounts.java

```

javac -d . BankAccounts.java
java javapractice.BankAccounts

```

7. Output Screen

```

befor transactions

Savings Account:
Account Number: 101
Account Holder: riya
Account Type: Savings
Balance: 10000.0

Current Account:
Account Number: 102
Account Holder: Rahul
Account Type: Current
Balance: 5000.0
savings account transactions
Deposited is 2000.0
Withdrawn is 1000.0
Interest Added: 550.0
current account transactions
Deposited is 2000.0
Withdrawn: 9000.0
amount transfer
Transferred: 3000.0
after transaction

Savings Account:

```

Account Number: 101
Account Holder: riya
Account Type: Savings
Balance: 8550.0

Current Account:
Account Number: 102
Account Holder: Rahul
Account Type: Current
Balance: 1000.0
final amount
Savings Account Balance: 8550.0
Current Account Balance: 1000.0

8. Balance Calculation

Savings: $10000 + 2000 - 1000 = 11000$
Interest = $11000 \times 5 / 100 = 550$
After interest = 11550
After transfer of 3000 = 8550

Current: $5000 + 2000 - 9000 + 3000 = 1000$
The 9000 withdrawal is permitted because the overdraft limit is 3000.

9. Result

The Bank Account Management System was successfully implemented in Java using classes and objects. The program demonstrates constructors, inheritance, method overriding, deposits, withdrawals, interest calculation, overdraft facility, fund transfer, and object creation successfully.